Cleaning up integer-class types

Document #:	P2393R0
Date:	2021-06-12
Project:	Programming Language C++
Audience:	LWG

Reply-to:	Tim Song <t.canens.cpp@gmail.com>

§ 1 Abstract

This paper revamps the specification and use of integer-class types to resolve a number of issues, including [LWG3366] and [LWG3376].

§ 2 Discussion

Integer-class types, introduced in [P1522R1], are implementation-defined class types that are supposed to "behave as integers do". Unfortunately, they are not yet required to do that, and the failure to do so leads to a number of issues in the library specification. For example:

— Two integer-class types are not required to have a `common_type`, which causes issues in various ranges components such as `zip_view` ([P2321R1]) and `join_view` (24.7.11.3 [range.join.iterator]) that uses `common_type_t` to determine the difference type of a range resulting from the composition of multiple ranges.

— Integer-class types are not required to be convertible or comparable to each other, so it is unclear how algorithms (such as `ranges::equal`) that may need to compare distances obtained from two ranges can even be implemented.

— The range of representable values of integer-class types, and their conversion to/from integer types, is ill-defined.

Additionally, there's a pervasive problem in the ranges and algorithms clauses in that the wording fails to take into account the fact that random access iterator operations are only required to work with the iterator's difference type, and not any other integer-like type. This was not a major issue in practice before C++20 because reasonable users (and even somewhat less reasonable ones) do not normally go out of the way to define deleted overloads for other integer types (or constrain their overload to require exactly the same type), but now that integer-class types can require explicit conversions, things are more problematic.

§ 2.1 Approach

First, tighten the specification of integer-class types to mandate support for what algorithms and range adaptors require, and clean up several library issues in the process.

— Specify that integer-class types are two's complement, just like built-in integers, and are wider than any built-in integer type. This resolves [LWG3366].

— Allow integer-class types to be non-class types, which resolves [LWG3376].

— Specify that two integer-like types always have a common type, and that common type is signed if both are signed, to support the use of `common_type_t` to compute the difference type of a range. (Only the signed case is considered because a) there is no corresponding use case for unsigned and b) the property doesn't hold for some built-in unsigned integer types that could be promoted to a signed type by integral promotion.)

— Specify that integer-class types are implicitly convertible to another integer-class type of the same signedness and equal or greater width; and are explicitly convertible to all integer-class types.

— Permit heterogeneous binary operations between integer-like types where one is implicitly convertible to the other; this allows arithmetic and comparison on the difference types of different ranges.

Next, clean up the ranges wording (again) to explicitly cast to the difference type where required.

Finally, add blanket wording in the algorithms clause that `i + n` and `i - n` for iterator `i` and integer-like type `n` in the wording behave as-if `n` is first cast to `i`'s difference type. This allows the spec to have things like `first1 + (last2 - first2)` without worrying about having to cast them (a fully correct implementation would still need to cast, however).

§ 3 Wording

This wording is relative to [N4885] after the application of [P2367R0].

1. Edit 23.3.4.4 [iterator.concept.winc], as indicated:

[Drafting note: Because *integer-class type* is only used in this subclause (the rest of the standard uses *integer-like*), I did not rename it. Cf. `enum class`.]

[Editor's note: P2321R2 § 5.4 also contains edits to this subclause. The wording below subsumes those edits and should be applied instead if both papers are adopted.]

2 A type `I` is an *integer-class* type if it is in a set of implementation-defined class types that behave as integer types do, as defined below. [*Note ?:* An integer-class type is not necessarily a class type. — *end note*]

3 The range of representable values of an integer-class type is the continuous set of values over which it is defined. The values 0 and 1 are part of the range of every integer-class type. If any negative numbers are part of the range, the type is a *signed-integer-class* type; otherwise, it is an *unsigned-integer-class* type. For any integer-class type, its range of representable values is either -2^{N-1} to $2^{N-1}-1$ (inclusive) for some integer N, in which case it is a *signed-integer-class* type, or 0 to $2^{N}-1$ (inclusive) for some integer N, in which case it is an *unsigned-integer-class* type. In both cases, N is called the *width* of the integer-class type. The

width of an integer-class type is greater than that of every integral type of the same signedness.

[Editor's note: Move paragraph 11 here to make *integer-like* available for use in subsequent wording.]

11 A type `I` other than *cv* `bool` is *integer-like* if it models `integral<I>` or if it is an integer-class type. An integer-like type I is *signed-integer-like* if it models `signed_integral<I>` or if it is a signed-integer-class type. An integer-like type `I` is *unsigned-integer-like* if it models `unsigned_integral<I>` or if it is an unsigned-integer-class type.

[Drafting note: The *unique* addition ensures that even if an implementation decides to provide two integer-class types of the same signedness and width, the result of binary operators are still well-defined.]

4 For every integer-class type `I`, let `B(I)` be a unique hypothetical extended integer type of the same signedness with the same width (6.8.2 [basic.fundamental]) as `I` the smallest width (6.8.2 [basic.fundamental]) capable of representing the same range of values. The width of `I` is equal to the width of `B(I)`. For every integral type `J`, let `B(J)` be the same type as `J`.

[Editor's note: Reorder paragraph 6 before 5.]

6 Expressions of integer-class type are explicitly convertible to any integer-likeintegral type, and implicitly convertible to any integer-class type of equal or greater width and the same signedness. Expressions of integral type are both implicitly and explicitly convertible to any integer-class type. Conversions between integral and integer-class types and between two integer-class types do not exit via an exception. The result of such a conversion is the unique value of the destination type that is congruent to the source modulo 2^N, where N is the width of the destination type.

5 Let `a` and `b` be an objectobjects of integer-class type `I`, let `b` be an object of integer-like type `I2` such that the expression `b` is implicitly convertible to `I`, let `x` and `y` be respectively objects of type `B(I)` and `B(I2)` as described above that represent the same values as `a` and `b` respectively, and let `c` be an lvalue of any integral type.

(5.1) — For every unary operator `@` for which the expression `@x` is well-formed, `@a` shall also be well-formed and have the same value, effects, and value category as `@x` provided that value is representable by `I`. If `@x` has type `bool`, so too does `@a`; if `@x` has type `B(I)`, then `@a` has type `I`.

(5.2) — For every assignment operator `@=` for which `c @= x` is well-formed, `c @= a` shall also be well-formed and shall have the same value and effects as `c @= x`. The expression `c @= a` shall be an lvalue referring to `c`.

(5.3) — For every non-assignment binary operator `@` for which `x @ y` and `y @ x` areis well-formed, `a @ b` and `b @ a` shall also be well-formed and shall have the same value, effects, and value category as `x @ y` and `y @ x` respectivelyprovided that value is representable by `I`. If `x @ y` or `y @ x` has type `bool`, so too does `a @ b` or `b @ a`,

respectively; if `x @ y` or `y @ x` has type `B(I)`, then `a @ b` or `b @ a`, respectively, has type `I`; if `x @ y` or `y @ x` has type `B(I2)`, then `a @ b` or `b @ a`, respectively, has type `I2`..

(5.4) — For every assignment operator `@=` for which `x @= y` is well-formed, `a @= b` shall also be well-formed and shall have the same value and effects as `x @= y`. The expression `a @= b` shall be an lvalue referring to `a`.

7 An expression `E` of integer-class type `I` is contextually convertible to `bool` as if by `bool(E != I(0))`.

8 All integer-class types model `regular` (18.6 [concepts.object]) and `totally_ordered` (18.5.4 [concept.totallyordered]).

9 A value-initialized object of integer-class type has value 0.

[Drafting note: There are some issues with the `numeric_limit` specialization: `digits` is defined in terms of `radix`, so it doesn't make sense to define the former but not the latter, and the definition of `digits` is also incorrect for signed types. Instead of trying to fix this piecemeal and maintain an ever-growing list, we can simply specify this in terms of `B(I)`.]

10 For every (possibly cv-qualified) integer-class type `I`, let `numeric_limits<B(I)>` be a hypothetical specialization that meets the requirements for `numeric_limit` specializations for arithmetic types (17.3.5 [numeric.limits]). `numeric_limits<I>` is specialized such that each static data member *`m`* has the same value as `numeric_limits<B(I)>::`*`m`*, and each static member function *`f`* returns `I(numeric_limits<B(I)>::`*`f`*`())`.:

(10.1) — ~~`numeric_limits<I>::is_specialized` is `true`;~~

(10.2) — ~~`numeric_limits<I>::is_signed` is `true` if and only if `I` is a signed-integer-class type;~~

(10.3) — ~~`numeric_limits<I>::is_integer` is `true`;~~

(10.4) — ~~`numeric_limits<I>::is_exact` is `true`;~~

(10.5) — ~~`numeric_limits<I>::digits` is equal to the width of the integer-class type;~~

(10.6) — ~~`numeric_limits<I>::digits10` is equal to `static_cast<int>(digits * log10(2))`; and~~

(10.7) — ~~`numeric_limits<I>::min()` and `numeric_limits<I>::max()` return the lowest and highest representable values of `I`, respectively, and `numeric_limits<I>::lowest()` returns `numeric_limits<I>::min()`.~~

? For any two integer-like types `I1` and `I2`, at least one of which is an integer-class type, `common_type_t<I1, I2>` denotes an integer-class type whose width is not less than that of `I1` or `I2`. If both `I1` and `I2` are signed-integer-like types, then `common_type_t<I1, I2>` is also a signed-integer-like type.

12 *`is-integer-like`*`<I>` is true if and only if I is an integer-like type. *`is-signed-integer-like`*`<I>` is true if and only if I is a signed-integer-like type.

2. Edit 24.5.4.2 [range.subrange.ctor] p6 as indicated:

```
template<not-same-as<subrange> R>
  requires borrowed_range<R> &&
           convertible-to-non-slicing<iterator_t<R>, I> &&
           convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

6 Effects: Equivalent to:

(6.1) — If *StoreSize* is `true`, `subrange(r, static_cast<decltype(size_)>(ranges::size(r)))`.

(6.2) — Otherwise, `subrange(ranges::begin(r), ranges::end(r))`.

3. Edit 24.7.7.1 [range.take.overview] p2 as indicated:

2 The name `views::take` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

(2.1) — If T is a specialization of `ranges::empty_view` (24.6.2.2 [range.empty.view]), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.

(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is

(2.2.1) — […]

then `T(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distancesize(E), F))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::take_view(E, F)`.

4. Edit 24.7.7.2 [range.take.view] as indicated:

```
namespace std::ranges {
  template<view V>
  class take_view : public view_interface<take_view<V>> {

    // [...]

    constexpr auto begin() requires (!simple-view<V>) {
      if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
          return ranges::begin(base_);
        else {
          auto sz = range_difference_t<V>(size());
          return counted_iterator((ranges::begin(base_), sz);
        }
      } else
```

```
      return counted_iterator(ranges::begin(base_), count_);
    }

    constexpr auto begin() const requires range<const V> {
      if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>)
          return ranges::begin(base_);
        else {
          auto sz = range_difference_t<const V>(size());
          return counted_iterator(ranges::begin(base_), sz);
        }
      } else
        return counted_iterator(ranges::begin(base_), count_);
    }

    constexpr auto end() requires (!simple-view<V>) {
      if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
          return ranges::begin(base_) + range_difference_t<V>(size());
        else
          return default_sentinel;
      } else
        return sentinel<false>{ranges::end(base_)};
    }

    constexpr auto end() const requires range<const V> {
      if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>)
          return ranges::begin(base_) + range_difference_t<const V>(size());
        else
          return default_sentinel;
      } else
        return sentinel<true>{ranges::end(base_)};
    }
    // [...]
  };

  template<class R>
    take_view(R&&, range_difference_t<R>)
      -> take_view<views::all_t<R>>;
}
```

5. Edit 24.7.9.1 [range.drop.overview] p2 as indicated:

2 The name `views::drop` denotes a range adaptor object (24.7.2 [range.adaptor.object]). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

(2.1) — If T is a specialization of `ranges::empty_view` (24.6.2.2 [range.empty.view]), then `((void) F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately sequenced.

(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is

(2.2.1) — […]

then `T(ranges::begin(E) + std::min<D>(ranges::distancesize(E), F), ranges::end(E))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::drop_view(E, F)`.

6. Edit 24.7.13 [range.counted] p2 as indicated:

2 The name `views::counted` denotes a customization point object (16.3.3.3.6 [customization.point.object]). Let `E` and `F` be expressions, let `T` be `decay_t<decltype((E))>`, and let `D` be `iter_difference_t<T>`. If `decltype((F))` does not model `convertible_to<D>`, `views::counted(E, F)` is ill-formed.

[*Note 1:* This case can result in substitution failure when `views::counted(E, F)` appears in the immediate context of a template instantiation. — *end note*]

Otherwise, `views::counted(E, F)` is expression-equivalent to:

(2.1) — If `T` models `contiguous_iterator`, then `span(to_address(E), static_cast<size_t>(static_cast<D>(F)))`.

(2.2) — Otherwise, if `T` models `random_access_iterator`, then `subrange(E, E + static_cast<D>(F))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `subrange(counted_iterator(E, F), default_sentinel)`.

7. Add the following to 25.2 [algorithms.requirements] after p12:

? In the description of the algorithms, given an iterator `a` whose difference type is `D`, and an expression `n` of integer-like type other than *cv* `D`, the semantics of `a + n` and `a - n` are, respectively, those of `a + D(n)` and `a - D(n)`.

§ 4 References

[LWG3366] Casey Carter. Narrowing conversions between integer and integer-class types. https://wg21.link/lwg3366

[LWG3376] Jonathan Wakely. “integer-like class type” is too restrictive. https://wg21.link/lwg3376

[N4885] Thomas Köppe. 2021-03-17. Working Draft, Standard for Programming Language C++. https://wg21.link/n4885

[P1522R1] Eric Niebler. 2019-07-28. Iterator Difference Type and Integer Overflow. https://wg21.link/p1522r1

[P2321R1] Tim Song. 2021-04-11. zip.
https://wg21.link/p2321r1

[P2367R0] Tim Song. 2021-04-30. Remove misuses of list-initialization from Clause 24.
https://wg21.link/p2367r0